EMPLOYEE ATTRITION AND INTERVENTION RECOMMENDATION SYSTEM



Team 13

Assessment 2 Data Analysis Project : Report

Statistical Thinking for Data Science

TD School

University of Technology Sydney

Table of Contents

Table of Contents

Executive Summary: 3

1) Introduction: 3

2) Methods: 3

3) Results: 4

3.1. Exploratory Data Analysis 4

3.2. Logistic Regression: 5

3.3. Random Forest: 5

3.4. XGBoost: 6

3.5. Model Comparison: 6

3.6. Feature Importance and SHAP Analysis: 7

3.7. Risk Stratification: 7

4) Discussion: 8

5) Conclusion: 8

6) References: 9

Lists of Figures

Figure 1–2. Overall attrition distribution and attrition rate by job role 3

Figure 3–4. Attrition rate by overtime status and monthly income distribution 3

Figure 5–6. Age distribution by attrition and attrition by department and business travel 4

Figure 7–8. Logistic Regression confusion matrix, ROC curve, and coefficient plot 4

Figure 9–10. Random Forest confusion matrix, ROC curve, and feature importance plot 5

Figure 11–12. XGBoost confusion matrix, ROC curve, and feature gain plot 5

Figure 13–14. Grouped bar chart comparing model metrics and ROC curve overlay 6

Figure 15–16. SHAP beeswarm plot and cross-method feature importance heatmap 6

Figure 17. XGBoost risk score distribution test set 7

Executive Summary:

Employee attrition, which refers to the voluntary departure of employees, costs organisations 50–200% of an employee's annual salary in recruitment, onboarding, and lost productivity. This report analyses the **IBM HR Analytics dataset** (1,470 employees, 35 variables) to identify the principal drivers of attrition and build a predictive model capable of flagging at-risk staff before they resign. The overall attrition rate is **16.1%** (237 of 1,470). Overtime is the strongest categorical predictor, meaning employees working overtime leave at **30.5%** versus **10.4%** for those who do not. Sales Representatives exhibit the highest role-level attrition at **39.8%**, and leavers earn on average **$2,046/month less** than stayers. Three classifiers were compared; **XGBoost delivered the highest recall (57.45%) and ROC-AUC (0.741)**. SHAP analysis identifies StockOptionLevel, JobSatisfaction, and EnvironmentSatisfaction as the most influential predictors. Four targeted interventions are recommended: auditing overtime, restructuring Sales Representative compensation, implementing early-career onboarding, and extending equity grants to front-line staff.

1) Introduction:

Employee attrition is one of the most financially and operationally significant challenge in human resource management. Beyond direct cost, high turnover erodes institutional knowledge, disrupts team cohesion, and signals underlying dysfunction in leadership, workload, or career opportunity. Traditional HR responses are reactive, where exit interviews occur only after a resignation. A predictive, data-driven approach offers a fundamentally different posture: analysing existing employee records to identify who is most likely to leave before departure occurs, enabling timely and targeted retention action.

Research Topic: Identifying and predicting the drivers of voluntary employee attrition using structured HR data.

Aim: To determine the key demographic, occupational, and satisfaction-related predictors of attrition in the IBM HR Analytics dataset and develop predictive models to flag at-risk employees.

Objectives:

1. Conduct EDA to identify variables most strongly associated with attrition.
2. Pre-process the data and address class imbalance using SMOTE
3. Build a Logistic Regression model as the primary interpretable regression model.
4. Benchmark against Random Forest and XGBoost; select the best-performing model.
5. Apply SHAP values to explain individual feature contributions.
6. Translate findings into evidence-based HR recommendations.

2) Methods:

Dataset: The IBM HR Analytics Employee Attrition & Performance dataset (Kaggle) contains 1,470 employee records across 35 variables covering demographics, job characteristics, compensation, and attitudinal satisfaction scales. The binary target is *Attrition* (Yes/No). No missing values were present.

Pre-processing: Four zero-variance or identifier columns were removed (EmployeeCount, EmployeeNumber,StandardHours, Over18). Seven categorical variables were label-encoded; numerical features were standardised with StandardScaler for Logistic Regression. Data were split 80/20 with stratification (train n = 1,176; test n = 294). To address class imbalance (83.9% stayed; 16.1% left), SMOTE was applied to the training set, producing 1,972 balanced records. The test set retained its natural distribution.

Models: Three supervised classifiers were trained and evaluated:

(1) **Logistic Regression** (max_iter=1000): interpretable baseline providing coefficient-level insight.

(2) **Random Forest** (300 trees) ensemble capturing non-linear interactions.

(3) **XGBoost** (300 estimators, depth=4, lr=0.05) gradient-boosted trees with regularisation.

Performance was measured by accuracy, precision, recall, F1, and ROC-AUC, with recall and AUC prioritised given the cost asymmetry of missing a true leaver. Five-fold stratified cross-validation assessed model stability. Feature importance was examined via LR coefficients, RF Gini scores, XGBoost gain, and SHAP values.

Characteristic	Value
Total records	1470
Variables (after cleaning)	31
Missing values	0
Attrition=Yes (Left)	237(16.1%)
Attrition=No (Stayed)	1233(83.9%)
Train/Test split	1176/294 (stratified 80/20)
Training set after SMOTE	1972 (balanced 50/50)
Mean income – Stayed/Left	$6833/ $4787 (gap $2046/month)

Table 1. IBM HR Analytics dataset summary statistics.

3) Results:

3.1. Exploratory Data Analysis

The dataset contains 237 leavers (16.1%) and 1,233 stayers (83.9%), confirming a class-imbalanced distribution (Figure 1).
Attrition varies substantially across job roles, working conditions, age groups, and compensation levels.

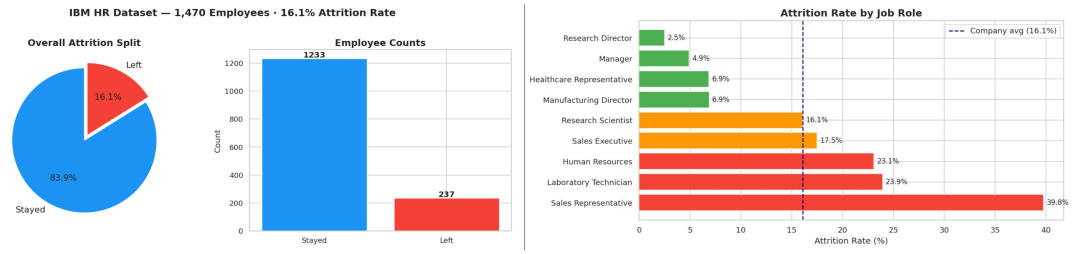


Figure 1–2. Left (Fig. 1): overall attrition distribution (16.1% left, 83.9% stayed). Right (Fig. 2): attrition rate by job role Sales Representatives lead at 39.8%,
followed by Laboratory Technicians (23.9%) and Human Resources (23.1%).

Figure 3 shows that employees working overtime leave at **30.5%** versus **10.4%** without overtime a three-fold difference and the strongest categorical predictor in the dataset. Figure 4 illustrates the income gap: leavers' mean monthly income ($4,787) is $2,046 below stayers ($6,833), with consistently lower distributions across all quantiles.

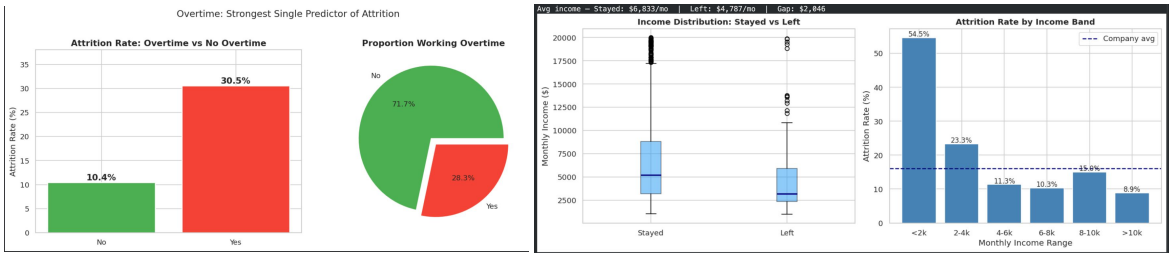


Figure 3–4. Left (Fig. 3): attrition rate by overtime status (30.5% with vs 10.4% without). Right (Fig. 4): monthly income distribution leavers earn substantially less at every quartile.

Age analysis (Figure 5) reveals that the 18–25 cohort leaves at **35.8%** nearly four times the 36–45 rate pointing to a failure in early-career retention. Department-level analysis (Figure 6) confirms Sales as the highest-attrition department, with frequent business travellers also at elevated risk.

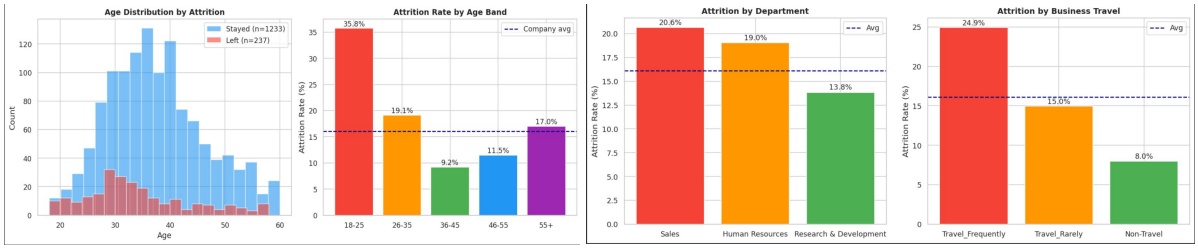


Figure 5–6. Left (Fig. 5): age distribution by attrition 18–25year-olds leave at 35.8%. Right (Fig. 6): attrition by department and business travel frequency.

3.2. Logistic Regression:

Logistic Regression achieved an **accuracy of 80.0%**, **recall of 51.1%** (Left class), **precision of 40.0%**, **F1 of 0.449**, and **ROC-AUC of 0.718** on the test set. Five-fold cross-validated AUC was **0.833 ± 0.018**, confirming stable generalisation. Figure 7 shows the confusion matrix and ROC curve; Figure 8 presents the top coefficient magnitudes, identifying StockOptionLevel and JobSatisfaction as dominant predictors.

Class	Precision	Recall	F1	Support
Stayed	0.90	0.85	0.88	247
Left	0.40	0.51	0.45	47
Accuracy			0.80	294
Macro avg	0.65	0.68	0.66	294

Table 2. Logistic Regression classification report test set (n = 294).

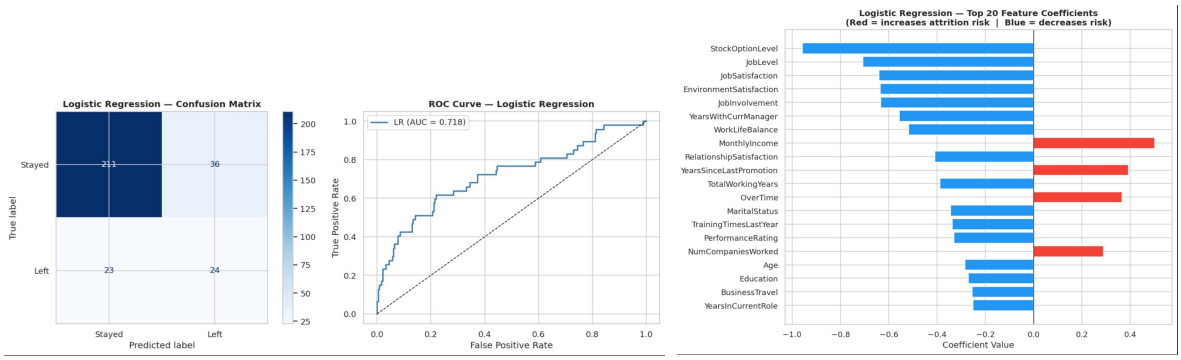


Figure 7–8. Left (Fig. 7): Logistic Regression confusion matrix and ROC curve (AUC = 0.718). Right (Fig. 8): LR coefficient plot StockOptionLevel and JobSatisfaction have the largest absolute coefficients.

3.3. Random Forest:

Random Forest is an ensemble method that builds many decision trees in parallel, with each trained on a random bootstrap sample of the data using a random subset of features. Final predictions are based on aggregated votes across all trees. This parallel randomisation, known as bagging, reduces variance and captures non-linear feature interactions that Logistic Regression cannot model. Feature importance is measured by mean Gini impurity reduction across all splits, providing a stable predictor ranking. Trained with 300 estimators on the unscaled SMOTE data, Random Forest achieved a cross-validated AUC of 0.967 ± 0.010. However, its test-set AUC (0.735) and recall (40.4%) were lower, indicating overfitting to the synthetic training data.

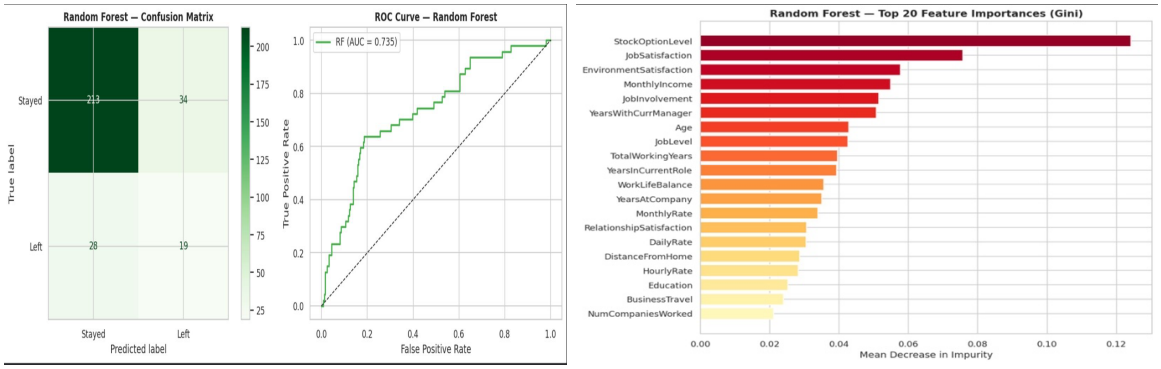


Figure 9–10. Left (Fig. 9): Random Forest confusion matrix and ROC curve (AUC = 0.735). Right (Fig. 10): Random Forest feature importance plot top predictors ranked by mean Gini impurity reduction across 300 trees.

3.4. XGBoost:

XGBoost builds trees sequentially, with each new tree fitted to the residual errors of the ensemble so far, guided by the gradient of a differentiable loss function. This boosting strategy, unlike Random Forest's parallel bagging, progressively corrects mistakes and achieves strong discriminative power on imbalanced tabular data. Built-in L1/L2 regularisation and early stopping help reduce overfitting. Trained with 300 estimators, max_depth=4, and learning_rate=0.05, XGBoost achieved the highest test-set recall (57.5%) and ROC-AUC (0.741), correctly flagging more true leavers than either alternative. It was selected as the primary operational attrition model.

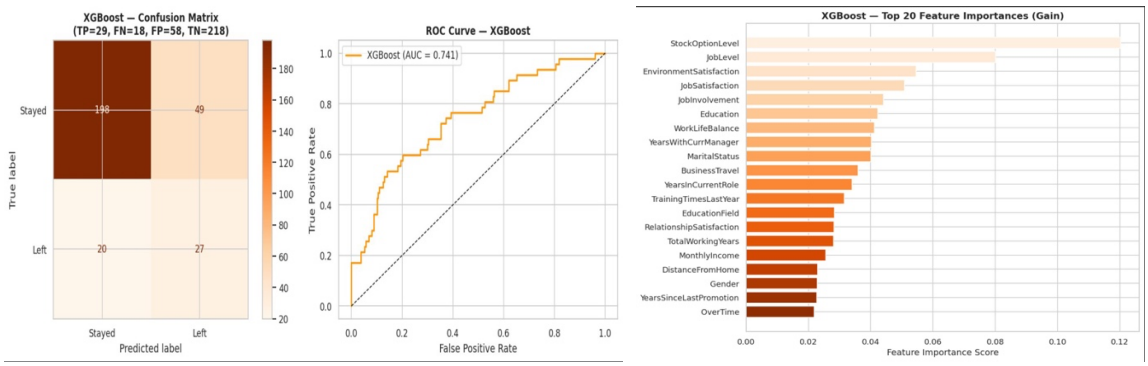


Figure 11–12. Left (Fig. 11): XGBoost confusion matrix and ROC curve (AUC = 0.741). Right (Fig. 12): XGBoost feature gain plot top predictors ranked by gain from gradient-boosted trees (300 estimators, max_depth=4, learning_rate=0.05).

3.5. Model Comparison:

Table 3 compares all three models on the held-out test set. **XGBoost achieves the highest recall (57.5%) and ROC-AUC (0.741)**, making it the preferred operational model. Logistic Regression achieves the highest accuracy (80.0%) but lower recall (51.1%). Random Forest's cross-validated AUC (0.967 ± 0.010) substantially exceeds its test AUC (0.735), indicating overfitting to the SMOTE-augmented training set.

Model	Accuracy	Precision	Recall	F1	ROC-AUC
Logistic Rergression	**0.799**	**0.400**	**0.511**	**0.449**	**0.718**
Random Forest	**0.789**	**0.359**	**0.404**	**0.380**	**0.735**
XGBoost	**0.765**	**0.355**	**0.575**	**0.439**	**0.741**

Table 3. Model performance on the held-out test set (n = 294). XGBoost leads on recall and ROC-AUC.

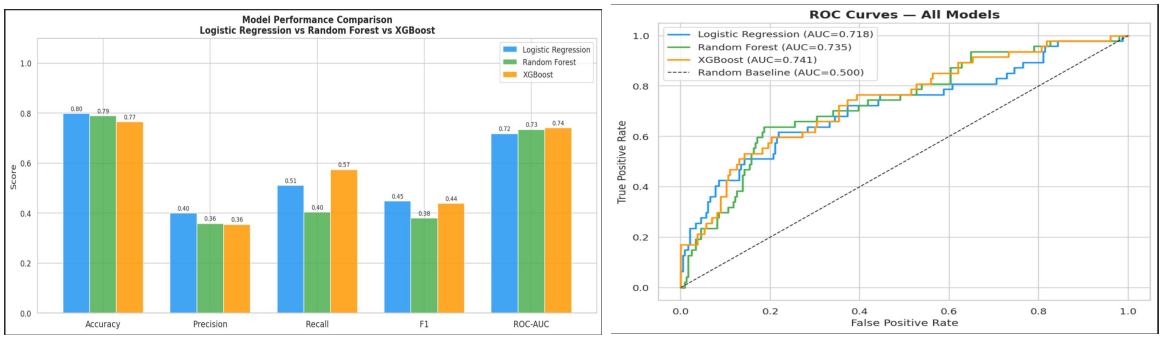


Figure 9–10. Left (Fig. 9): grouped bar chart comparing all metrics across three models. Right (Fig. 10): overlay of all three ROC curves XGBoost (AUC 0.741),
RF (0.735), LR (0.718).

3.6. Feature Importance and SHAP Analysis:

Normalised importance scores (Table 4) show **StockOptionLevel** ranking first across all four methods (LR, RF, XGBoost, SHAP = 1.000). JobSatisfaction, EnvironmentSatisfaction, and JobInvolvement consistently follow, reflecting the primacy of intrinsic workplace quality. OverTime achieves a SHAP importance of 0.387, corroborating the EDA findings. Figure 11 presents the SHAP beeswarm plot; Figure 12 shows the cross-method feature comparison.

Feature	LR IcoefI	Rand. Forest	XGBoost	SHAP
StockOptionLevel	**1.000**	**1.000**	**1.000**	**1.000**
JobSatisfaction	**0.670**	**0.610**	**0.423**	**0.718**
EnvironmentSatisfaction	**0.663**	**0.465**	**0.453**	**0.617**
JobInvolvement	**0.661**	**0.415**	**0.366**	**0.538**
YearsWithCurrManager	**0.580**	**0.409**	**0.334**	**0.481**
MonthlyIncome	**0.524**	**0.442**	**-**	**0.392**
OverTime	**0.383**	**-**	**-**	**0.387**
WorkLifeBalance	**0.541**	**0.287**	**0.342**	**0.391**

Table 4. Normalised feature importance (0–1) across all methods. "—" = feature not in top-ranked set for that method.

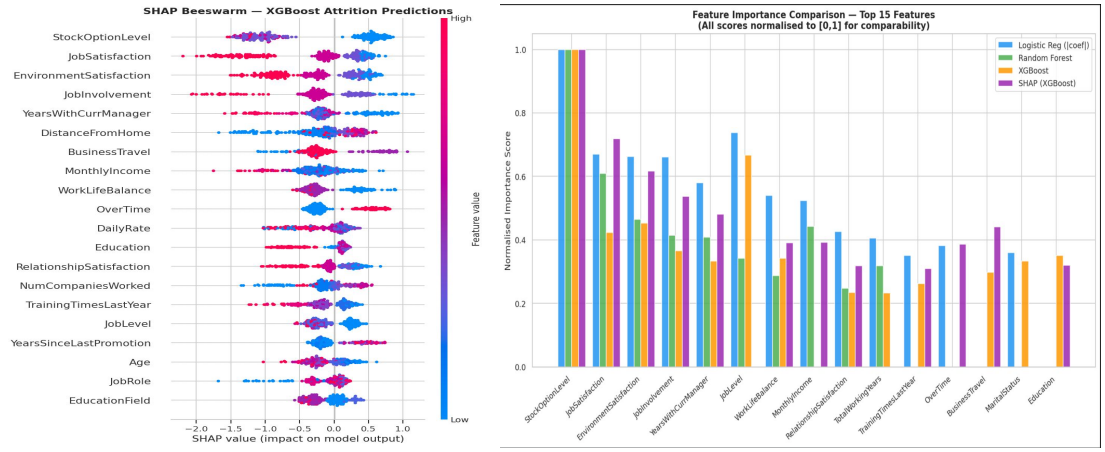


Figure 11–12. Left (Fig. 11): SHAP beeswarm plot red = high feature value, blue = low; ordered by mean |SHAP|. Right (Fig.12): cross-method feature importance heatmap.

3.7. Risk Stratification:

XGBoost probability scores were used to assign each test-set employee a risk tier: **Low** (<0.30, n=192), **Medium** (0.30–0.60, n=41), **High** (>0.60, n=61). Figure 13 confirms that actual leavers (red) concentrate at higher risk scores, validating the model's discriminative power operationally.

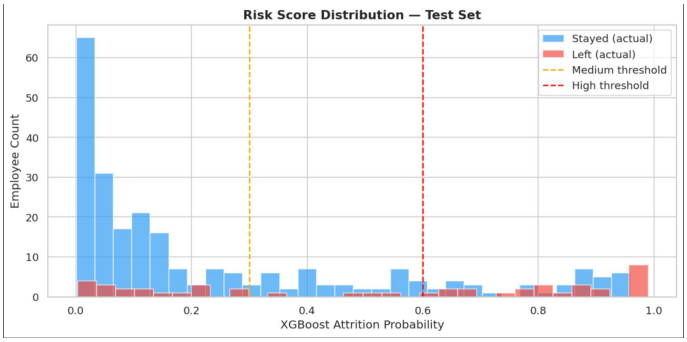


Figure 13. XGBoost risk score distribution test set. Employees who left (red) cluster at higher probability scores. Dashed lines mark Medium (0.30) and High (0.60) risk thresholds.

4) Discussion:

The findings confirm that attrition is multi-factorial. The three-fold overtime effect (30.5% vs 10.4%) is the most immediately actionable finding because overtime is a manageable working condition, unlike intrinsic attitudes that require sustained cultural change. The Sales Representative attrition rate (39.8%), combined with the $2,046 income gap, points to structural underpayment and insufficient career pathways in that function. The 18–25 cohort's 35.8% departure rate highlights a failure to retain early-career talent through onboarding and development.

Model selection: XGBoost is recommended as the operational model (recall 57.5%, AUC 0.741). In an attrition context, false negatives, meaning failing to flag a leaver, are costlier than false positives, making recall the priority metric. Logistic Regression remains valuable as a complementary explainability tool, as its coefficients (Figure 8) can be communicated directly to HR stakeholders without needing to explain ensemble model mechanics.

Limitations: The IBM dataset is synthetic, limiting external validity. Despite SMOTE, XGBoost still misses approximately 42% of true leavers. The Random Forest CV-to-test AUC gap (0.967 vs 0.735) suggests overfitting and warrants further hyperparameter tuning. Live deployment would require retraining on real data, regular recalibration, and ethical governance review to prevent discriminatory profiling.

5) Conclusion:

This study applied EDA and three machine-learning classifiers to the IBM HR Analytics dataset to understand and predict employee attrition. The overall attrition rate of **16.1%** is driven primarily by overtime work, job role (Sales Representatives at 39.8%), compensation gap ($2,046/month), and early career stage (18–25 cohort at 35.8%).XGBoost delivered the strongest predictive performance (ROC-AUC 0.741, recall 57.5%) and is recommended as the operational attrition-risk model. Four evidence-based interventions are proposed:

1. **Control overtime:** Audit and cap overtime in high-attrition roles; the three-fold attrition differential makes this the highest-priority action.
2. **Redesign Sales Rep retention:** Benchmark compensation to market rates; introduce 6–18month career pathway maps and quarterly pulse surveys.
3. **Early-career programme:** Implement 12-month structured onboarding with mentoring for all staff under 25, targeting the 35.8% departure rate.
4. **Expand equity participation:** Extend stock options to front-line staff with a two-year vesting cliff; StockOptionLevel is the single most important predictor across all methods.

6) References:

• . *IBM HR Analytics Employee Attrition & Performance* [Dataset]. Kaggle. https://www.kaggle.com/datasets/pavansubhasht/ibm-hr-analytics-attrition-dataset

Appendix A: Code

Setup

```
import warnings
warnings.filterwarnings('ignore')

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches
import seaborn as sns

from sklearn.model_selection import train_test_split, cross_val_score, StratifiedKFold
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import (
    classification_report, confusion_matrix, roc_auc_score,
    roc_curve, ConfusionMatrixDisplay, accuracy_score,
    precision_score, recall_score, f1_score
)
from sklearn.inspection import permutation_importance

from imblearn.over_sampling import SMOTE
import xgboost as xgb
import shap

# Global plot style
sns.set_theme(style='whitegrid', palette='muted')
plt.rcParams.update({'figure.dpi': 120, 'font.size': 11})

RANDOM_STATE = 42
print('Libraries loaded successfully.')
```

Load

```
df = pd.read_csv('/content/WA_Fn-UseC_-HR-Employee-Attrition.csv')



print(f'Shape: {df.shape}')
df.head()
```

```
print('=== Data Types ===')
print(df.dtypes.value_counts())
print('\n=== Missing Values ===')
print(df.isnull().sum().sum(), 'missing values total')
print('\n=== Target Distribution ===')
print(df['Attrition'].value_counts())
print(f"Attrition rate: {df['Attrition'].value_counts(normalize=True)['Yes']:.1%}")
```

```
=== Data Types ===
int64     26
object     9
Name: count, dtype: int64

=== Missing Values ===
0 missing values total

=== Target Distribution ===
Attrition
No     1233
Yes     237
Name: count, dtype: int64
Attrition rate: 16.1%
```

Pre Processing

```
# ── 3.1 Drop Constant / Non-informative Columns ──
drop_cols = ['EmployeeCount', 'StandardHours', 'Over18', 'EmployeeNumber',
             'AgeBand', 'IncomeBand']   # engineered EDA cols
df_model = df.drop(columns=drop_cols, errors='ignore').copy()

# Encode target
df_model['Attrition'] = (df_model['Attrition'] == 'Yes').astype(int)

print(f'Columns remaining: {df_model.shape[1]}')
df_model.head(3)
```

```
Columns remaining: 31
```

```
# ── 3.2 Label-Encode Categorical Features ──
cat_cols = df_model.select_dtypes(include='object').columns.tolist()
print(f'Categorical columns to encode ({len(cat_cols)}): {cat_cols}')

le_dict = {}
for col in cat_cols:
    le = LabelEncoder()
    df_model[col] = le.fit_transform(df_model[col])
    le_dict[col] = le

print('Encoding complete.')
df_model.dtypes.value_counts()
```

```
Categorical columns to encode (7): ['BusinessTravel', 'Department', 'EducationField', 'Gender', 'JobRole', 'MaritalStatus', 'OverTime']
```

```python
# ── 3.3 Train / Test Split ──
X = df_model.drop('Attrition', axis=1)
y = df_model['Attrition']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=RANDOM_STATE, stratify=y
)

print(f'Train: {X_train.shape}  |  Test: {X_test.shape}')
print(f'Train attrition rate: {y_train.mean():.1%}')
print(f'Test  attrition rate: {y_test.mean():.1%}')
```

```
Train: (1176, 30)  |  Test: (294, 30)
Train attrition rate: 16.2%
Test  attrition rate: 16.0%
```

```python
# ── 3.4 SMOTE Oversampling (fixes 84/16 class imbalance) ──
smote = SMOTE(random_state=RANDOM_STATE)
X_train_sm, y_train_sm = smote.fit_resample(X_train, y_train)

print(f'After SMOTE — Train shape: {X_train_sm.shape}')
print(f'Class distribution after SMOTE:\n{pd.Series(y_train_sm).value_counts()}')

# Scale for Logistic Regression
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train_sm)
X_test_scaled  = scaler.transform(X_test)
```

```
After SMOTE — Train shape: (1972, 30)
Class distribution after SMOTE:
Attrition
0    986
1    986
Name: count, dtype: int64
```

Logistic Regression

```
# ── 4.1 Train ──
lr = LogisticRegression(max_iter=1000, random_state=RANDOM_STATE)
lr.fit(X_train_scaled, y_train_sm)

y_pred_lr  = lr.predict(X_test_scaled)
y_prob_lr  = lr.predict_proba(X_test_scaled)[:, 1]

print('=== Logistic Regression — Classification Report ===')
print(classification_report(y_test, y_pred_lr, target_names=['Stayed', 'Left']))
```

```
# ── 4.4 Cross-Validation ──
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=RANDOM_STATE)

# Use unscaled SMOTE data here for a quick CV score
lr_cv = LogisticRegression(max_iter=1000, random_state=RANDOM_STATE)
cv_scores_lr = cross_val_score(lr_cv, X_train_sm, y_train_sm, cv=cv,
                               scoring='roc_auc', n_jobs=-1)
print(f'Logistic Regression CV AUC: {cv_scores_lr.mean():.3f} ± {cv_scores_lr.std():.3f}')
```

```
Logistic Regression CV AUC: 0.833 ± 0.018
```

Random Forest

```
# ── 5.1 Train ──
rf = RandomForestClassifier(
    n_estimators=300,
    max_depth=10,
    min_samples_leaf=5,
    class_weight='balanced',
    random_state=RANDOM_STATE,
    n_jobs=-1
)
rf.fit(X_train_sm, y_train_sm)

y_pred_rf = rf.predict(X_test)
y_prob_rf = rf.predict_proba(X_test)[:, 1]

print('=== Random Forest — Classification Report ===')
print(classification_report(y_test, y_pred_rf, target_names=['Stayed', 'Left']))
```

```
# ── 5.5 Cross-Validation ──
rf_cv_scores = cross_val_score(
    RandomForestClassifier(n_estimators=100, random_state=RANDOM_STATE, n_jobs=-1),
    X_train_sm, y_train_sm, cv=cv, scoring='roc_auc', n_jobs=-1
)
print(f'Random Forest CV AUC: {rf_cv_scores.mean():.3f} ± {rf_cv_scores.std():.3f}')
```

```
Random Forest CV AUC: 0.967 ± 0.010
```

XGBoost

```
# ── 6.1 Train ──
xgb_model = xgb.XGBClassifier(
    n_estimators=300,
    max_depth=4,
    learning_rate=0.05,
    subsample=0.8,
    colsample_bytree=0.8,
    scale_pos_weight=5,      # addresses class imbalance
    eval_metric='logloss',
    random_state=RANDOM_STATE,
    n_jobs=-1
)
xgb_model.fit(
    X_train_sm, y_train_sm,
    eval_set=[(X_test, y_test)],
    verbose=False
)

y_pred_xgb = xgb_model.predict(X_test)
y_prob_xgb = xgb_model.predict_proba(X_test)[:, 1]

print('=== XGBoost — Classification Report ===')
print(classification_report(y_test, y_pred_xgb, target_names=['Stayed', 'Left']))
```

Shap

```
# ── 7.1 Compute SHAP Values ──
explainer   = shap.TreeExplainer(xgb_model)
shap_values = explainer.shap_values(X_test)
print(f'SHAP values shape: {shap_values.shape}')
```

```
SHAP values shape: (294, 30)
```

```
# ── 7.2 SHAP Summary Plot (Beeswarm) ──
plt.figure(figsize=(10, 8))
shap.summary_plot(shap_values, X_test, plot_type='dot', max_display=20, show=False)
plt.title('SHAP Beeswarm — XGBoost Attrition Predictions', fontsize=13, fontweight='bold')
plt.tight_layout()
plt.show()
```